

Improved Factoring of RSA Modulus

Jiun-Ming Chen^{1,2}, Shouu-I Yu³, Yi Ou-Yang^{4,2}, Po-Han Wang³,
Chi-Hung Lin³, Po-Yi Huang⁵, Bo-Yin Yang⁶, Chi-Sung Lai¹
{jmchen, b94902065, b94901167}@ntu.edu.tw
{b92046, b93030}@csie.ntu.edu.tw
pyhuang@mail.ncku.edu.tw, by@moscito.org
laihcs@eembox.ncku.edu.tw

Abstract

In 1999, the 512-bit number of 155 digits taken from the RSA Challenge list was first factored by the General Number Field Sieve. This work was done on a supercomputer and about 300 PCs or workstations by 17 experts all over the world. The calendar time for the factorization was over 6 months. Based on the open source GGNFS, we improved its algorithms and implementations. Now the 512-bit RSA modulus can be factored within 3 days by the high-performance computing resource at National Taiwan University.

1 Introduction

Since its invention [12] thirty years ago, the RSA public-key cryptosystem has been widely deployed. It protects most of today's E-commerce on the internet. The security of RSA is based on the difficulty of factoring a large integer which is known to be the product of two unknown big primes. Currently bank transactions are protected by 1024-bit RSA standard, while careless users even take 512-bit RSA keys for SSL (Secure Socket Layer) handshake protocols.

The General Number Field Sieve (GNFS) [7] with two-decade history is still the most powerful large integer factoring algorithm. The best implementation of GNFS available on the internet is GGNFS [15],

which is an open source with more than twenty thousand lines of source code. It is good at factoring integers of 70 to 110 digits. We improved several parts of parameters, algorithms, and codes in the GGNFS to achieve our goal: Factoring 512-bit RSA modulus within a few days.

The complete GNFS algorithm consists of four major steps: *polynomial selection*, *sieving*, *matrix reduction*, and *square root*. We perform the first two and the last steps on the HP cluster, and the third step on the IBM p595 SMP (symmetric multi-processing) supercomputer at National Taiwan University (NTU).

Working on 50 cores of the HP cluster, the step of polynomial selection takes 24 hours, and the sieving step including *matbuild* process spends 76.2 hours. Note that the backend of the HP cluster consists of 424 cores (106 nodes), and these two steps can be highly parallelized. That is, on the full capacity of the HP cluster at NTU, the steps of polynomial selection and sieving cost no more than 20 hours definitely, though it is not executed in our actual experiment.

Since the step of matrix reduction can be partially parallelized only, we transfer our job to IBM p595 using 24 cores. The *matprune* (or *filtering*) takes 2.2 hours to reduce the size of the huge sparse matrix. Then the SMP supercomputer with OpenMP needs 37.5 hours to finish *matsolve*, i.e., solving system of equations by the Block Lanczos algorithm.

The last step, square root, takes 2.98 hours at the HP cluster. Therefore, with 424 cores of the cluster, the calendar time of factorization is less than 3 days.

Compared to the 6-month "world record" [4] set in 1999, the required resource and time of factoring 512-bit RSA modulus is dramatically reduced.

The GNFS algorithm in its current form is a very complicated beast. We will review GNFS briefly in Section 2 and the recent factoring records in Section 3. The platform of our factorization, high-performance computing facilities at National Taiwan University, is described in Section 4. Four major steps of GNFS are explained in Sections 5 to 8 respectively. In each of them, the flow of GGNFS, our work, and experiment results compared with [4] are introduced in this order. Finally we conclude in Section 9.

¹ Department of Electrical Engineering, National Cheng Kung University

² Department of Mathematics, National Taiwan University

³ Department of Computer Science and Information Engineering, National Taiwan University

⁴ Department of Electrical Engineering, National Taiwan University

⁵ Department of Mathematics, National Cheng Kung University

⁶ Institute of Information Science, Academia Sinica

* This work was supported in part by TWISC@NCKU, National Science Council under the Grants NSC 96-2219-E-006-009

* We are grateful to Computer and Information Networking Center, National Taiwan University for the support of high-performance computing facilities.

2 Overview of GNFS

Let N be the composite number we want to factor. Suppose $f(x)$ is an irreducible polynomial with integer coefficients of degree $d > 1$, and $f(m) \equiv 0 \pmod{N}$ for an integer m . Let θ be a complex root of $f(x)$, we will consider algebraic numbers in the ring $\mathbf{Z}[\theta]$.

Let ϕ be the ring homomorphism from $\mathbf{Z}[\theta]$ to $\mathbf{Z}/N\mathbf{Z}$ defined by $\phi(\theta) = m$. Suppose the set S consisting of integer pairs (a, b) is found such that

$$\prod_{(a,b) \in S} (a + b\theta) = \beta^2 \quad \text{and} \quad \prod_{(a,b) \in S} (a + bm) = y^2$$

with $\beta \in \mathbf{Z}[\theta]$ and $y \in \mathbf{Z}$. Applying ϕ , we have

$$\phi(\beta)^2 \equiv \phi\left(\prod_{(a,b) \in S} (a + b\theta)\right) \equiv \prod_{(a,b) \in S} (a + bm) \equiv y^2$$

modulo N . If $\gcd(\phi(\beta) - y, N) \neq 1$ or N (at least 50% chance), then we factor N successfully.

The first step of GNFS, *polynomial selection*, chooses two irreducible polynomials $f(x)$ and $g(x)$ with a common root $m \pmod{N}$ of small degrees d and e respectively. The polynomials have as many smooth values as possible over a given factor base.

The *sieving* step is the most time-consuming step, but its parallelization is easy. This step finds coprime pairs (a, b) such that both $b^d f(a/b)$ and $b^e g(a/b)$ are *smooth*, i.e., numbers with all prime factors smaller than a given upper bound. Such a pair (a, b) is called a *relation*. We record the factorization over the primes as an exponent vector. The purpose of this step is to search for sufficiently many relations.

Once enough relations are collected, a large matrix is constructed for the *matrix reduction* step. To find squares in both sides, a linear dependency over $\mathbf{GF}(2)$ among the rows of this matrix is required. The sparse system of equations is solved by Block Lanczos or Block Wiedemann algorithm.

The *square root* step computes the square root of an algebraic number of the form $\prod (a + b\theta)$, which is known to be a square in $\mathbf{Z}[\theta]$. Each $a + b\theta$ has smooth norm. This leads to a congruence $x^2 \equiv y^2 \pmod{N}$. If $\gcd(x - y, N)$ is trivial, we go back to the step of matrix reduction to find another linear dependency.

3 Factoring Records

Representative factoring records such as RSA-140, RSA-155, RSA-200, and RSA-640 [14] by the GNFS are reviewed here, as well as the factoring of $2^{1039} - 1$, which is the best achievement of the Special Number Field Sieve (SNFS) so far [1].

On February 2, 1999, the 140-digit number RSA-140 was factored by 9 experts with GNFS [3]. It was the second time that GNFS appeared in the world factoring records, while the factoring of RSA-130 was the first. The amount of spent computer time was

prudently estimated to be equivalent to 2000 MIPS years. Polynomial selection was improved by Peter Montgomery and Brian Murphy. It took four weeks on four 250 MHz SGI Origin 2000 processors. Sieving was done on 125 workstations (175Hz on average) and 60 PCs (300Hz on average) within one month. Matrix reduction took almost 12 days and 810 Mbytes of central memory on the Cray C916.

On August 22, 1999, the 155-digit, 512-bits number RSA-155 was factored by 17 experts [4]. The amount of computer time spent on this record is estimated to be equivalent to 8000 MIPS years. Polynomial selection took approximately 0.40 CPU years on a 250 MHz SGI Origin 2000 processor. Sieving was done on about 160 175-400 MHz SGI and Sun workstations, on 8 300 MHz SGI Origin 2000 processors, on about 120 300-450 MHz Pentium II PCs, and on 4 500 MHz Digital/Compaq boxes. The calendar time of sieving was 3.5 months. Matrix reduction took 9.5 days and 2 Gbytes of central memory on the Cray C916.

On May 9, 2005, the 200-digit, 663-bits number, RSA-200 was factored [14]. It is still an open record of GNFS today. Sieving was done on a variety of machines, estimated that it would take 55 years on a single 2.2 GHz Opteron CPU. For matrix solving, Block Lanczos was replaced by Block Wiedemann.

On November 2, 2005, the 640-bits number, RSA-640 was factored by a German team who took the last RSA challenge prize [14]. Sieving was done on 80 2.2 GHz Opteron CPUs and took 3 months. The factorization without polynomial selection took 5 months. The matrix step was performed on a cluster of 80 2.2 GHz Opterons connected via a Gigabit network and took 1.5 months.

On May 21, 2007, the kilobit number $2^{1039} - 1$ was factored by five experts with SNFS [1]. Although the small factor 5080711 was already known, SNFS could not take advantage of it. For SNFS, polynomial selection took no effort. They spent 6 months for sieving on various clusters and PCs. Total sieving time was scaled to about 95 Pentium D (3.0GHz) years. During matrix reduction, 7 days were paid for filtering, and Block Wiedemann cost 69 days.

For GNFS or SNFS, the *sieving* step always needs the most total computational time. But the *matrix reduction* step constitutes the bottleneck of large factorization efforts, since it does not allow the same level of parallelization as sieving or polynomial selection. This phenomenon was also demonstrated by our experiment at NTU, because matrix reduction takes more than half of the elapsed time of factoring.

Before the record of [1], matrix reduction could be only carried out on a cluster at a single location. The work of [1] differs from previous ones in the way that Block Wiedemann was done as four independent jobs in parallel on different clusters at various locations.

4 Experiment Environment

Two supercomputers at NTU for our factorization of 512-bit RSA modulus are described as follows.

1. The IBM p595 SMP, with 64 Power5+ 1.9MHz CPUs and 256GB RAM, runs the AIX 5.3 operating system. Programs suitable for running on the IBM p595 are those which are parallelized with OpenMP or require a lot of memory.
2. The HP cluster consists of 106 nodes connected by a Voltaire 288 DDR switch. Each node, which runs the Redhat 4 u3 AS kernel 2.6.9 operating system and has 4GB memory, is a HP ProLiant DL Server with two Woodcrest dual core 3.0GHz CPUs. It is suitable for running MPI programs.

5 Polynomial Selection

In this section, we briefly describe the polynomial selection algorithm, how GGNFS [15] realizes those algorithms, what we have done to improve it, and the experimental results.

5.1 Algorithm Description

The aim of polynomial selection is to choose “good” polynomials which generate many smooth values. They reduce the time consuming in sieving step and matrix reduction step. So there is a trade-off between polynomial search time and its saving in sieving time. The best reported results were achieved by the method of Thorsten Kleinjung [6], which finds polynomial pairs with $\deg(f) = d$ and $\deg(g) = 1$.

A polynomial’s “goodness” is determined by its *yield* [10], that is, the number of smooth values it produces for a given smoothness bound and in a given range. A polynomial yield can be determined without sieving. Let A be a region we will sieve and B_1, B_2 be the smoothness bounds of f, g , then the yield has an approximation as

$$\frac{6}{\pi^2} \int_A \rho\left(\frac{\log(f(x,y)) + \alpha_1}{\log(B_1)}\right) \rho\left(\frac{\log(g(x,y) + \alpha_2)}{\log(B_2)}\right) dx dy \quad (5.1.1)$$

where $\rho(x)$ denotes Dickman’s function which is roughly the probability that the largest prime factor of a natural number n is at most $n^{1/x}$.

Since this expression is difficult to compute, we need simpler approximations for implementation and research. In this case, g is of degree one, so its yield is less important. Hence, the yield of f has a further simplification $\alpha(f) + \beta(f)$, where $\alpha(f)$ is called the *root property* defined by

$$\alpha(f) = \sum_p \left(1 - r(f, p) \frac{p}{p+1}\right) \frac{\log p}{p-1} \quad (5.1.2)$$

where the sum ranges over small primes p and $r(f, p)$ denotes the number of linear factors of $f(x) \pmod{p}$. The term $r(f, p)$ appears because it provides useful prime ideal in sieving step.

The coefficient sizes of the polynomial f are also important, since smaller coefficients produce smaller numbers which are more likely smooth. The term $\beta(f)$ called *size property* defined by

$$\beta(f) = \frac{1}{2} \log \left(\int_A y^d f(x/y) dx dy \right) \quad (5.1.3)$$

is basically determined by the size of coefficients.

We briefly describe Kleinjung’s algorithm [6]: He finds a smart method that chooses suitable parameters, then generates one degree-one polynomial $g(x)$ and a family of polynomials $f_i(x)$ which have a common root $m \pmod{N}$. Those f_i have a very good property that we can check their size property $\beta(f_i)$ in short time. Roughly speaking, n polynomials are checked in time $O(n^{1/2})$. It is the main contribution of [6].

Here their size is the *skewness* [6]. A polynomial’s skewness size is basically determined by its first three coefficients a_5, a_4, a_3 . We consider $f'_i(x) = f_i(x) + g(x)h(x)$, where $h(x)$ is of degree one without large coefficients. Changing a_2, a_1, a_0 of f_i only change its skewness size property slightly. The size properties of all f'_i are still good. We search for those with good root property.

The above method is repeated to generate many polynomials with good yield. Kleinjung recommends $60|a_5$ for good projective root property. The best of these polynomials is used for sieving.

The branch of GGNFS doing polynomial selection is called *Pol5*. It was originally written by Thorsten Kleinjung according to [6]. *Pol5* is divided into two parts: *pol51m0* and *pol51opt*. They are introduced in the following paragraphs.

5.2 Part 1: pol51m0

The first part of polynomial selection produces a lot of (a_5, p, d) triples. The polynomials generated by these triples have smaller a_3 coefficients. It replaces the base- m method in Murphy’s algorithm [10].

Probably because it is the distillation of the whole program, the code itself is nearly unreadable. Uncommented code, architecture dependent assembly code, and badly-named or German variables are throughout the program. So we decided not to try to fully understand the code, but to examine its correctness, and try to find a better parameter setting.

The only tunable parameter in *pol51m0* is *NORM_MAX* (NM). The main use of this parameter is to determine what (a_5, p, d) triple will be sent to the next part. For each triple the following property will be computed:

$$\sup(f, s) = \max_i \left| a_i s^{i - \frac{d}{2}} \right| \quad (5.2.1)$$

$$\sup(f) = \min_{s > 0} \sup(f, s) \quad (5.2.2)$$

$\sup(f)$ is a approximation of size property, and s is a variable of skewness. Those triples with $\sup(f) > \text{NM}$ (too bad in size-property) are not sent to the next part.

After several examinations, we find out that if we set NM too high, then the whole process slows down significantly. On the other hand, we will get too few triples if we set it too low. Therefore, we decide not to change the value used in GGNFS.

NORM_MAX	Time (hrs)
7.12×10^{23}	1081.1
1.00×10^{24}	1521.3

$$a_5\text{-max} = 1 \times 10^8$$

$$\text{NORM_MAX1 (NM1)} = 8.75 \times 10^{21}$$

$$\text{NORM_MAX2 (NM2)} = 1.00 \times 10^{19}$$

5.3 Part 2: pol51opt

The second part of Pol5 expands a (a_5, p, d) triple to a complete polynomial, and uses those methods provided by Brian Murphy to do the optimization.

This part of the code is much more readable, so we can do some parameter selection with more detailed analysis. The procedure of the program contains the follow steps:

- 1) Find a skewness s which fits (5.3.1) most, do some translations (5.3.2) and rotations (5.3.3) (5.3.4) to fine-tune the size-property.

$$\sup_{L^2}(f, s) = \sqrt{\int_0^1 \int_0^1 \left(f\left(\frac{sx}{y}\right) \times \left(\frac{y}{\sqrt{s}}\right)^d \right)^2 dx dy} \quad (5.3.1)$$

$$f_i(x) = f(x - k) \quad (5.3.2)$$

$$f_p(x) = f(x) + P(x) \times (px - d) \quad (5.3.3)$$

$$P(x) = \pm \left(\frac{a_1}{d} \right) x + \left(\frac{a_0}{d} \right) \quad (5.3.4)$$

- 2) Root-Sieve: on (j_1, j_0) -plane, for each (j_1, j_0) , add $(j_1 x - j_0)(px - d)$ to the original polynomial, and use the following formula to compute the alpha value (root-property):

$$\sum_{p \text{ small}} \left(1 - r(f, p) \frac{p}{p+1} \right) \frac{\log(p)}{p-1} \quad (5.3.5)$$

- 3) Find a new skewness based on Murphy_E (5.1.1) (both size and root properties are considered), and do some translations (5.3.2) to fine-tune it again.

There are 3 parameters which can be tuned in pol51opt – NM1, NM2, and Murphy_E. NM1 decides how many polynomials will enter the root-sieve step. Those polynomials with L^2 -norm (5.3.1) smaller than NM1 will pass. Because root sieve is the most time-consuming step, our goal is to minimize NM1 while not to lower the capability to find polynomials with good root property. After some data collection and calculation, we conclude that it is fine to lower the NM1 from 8.75×10^{21} to 4.2×10^{21} without missing polynomials with good root property. It definitely benefits the performance.

In the following tables, RSA-155 stands for the RSA challenging number of 512 bits; D155_xx are numbers multiplied by two randomly generated 256-bit prime numbers.

Number	NORM_MAX1	Time (hrs)
D155_01	8.75×10^{21}	1081.1
D155_01	4.20×10^{21}	293.7
D155_02	8.75×10^{21}	969.7
D155_02	4.20×10^{21}	266.2

$$a_5\text{-max} = 1 \times 10^8, \text{NM2} = 1.00 \times 10^{19}$$

The Murphy_E parameter controls the output of polynomial candidates. It is not necessary to change it, since we want the best only.

NM2 is a threshold of how many polynomials' Murphy_E value will be actually computed. Although the computation of Murphy_E itself requires much time, it is not as critical as NM1 since there are fewer polynomials now. The selections of NM1 and NM2 are relative, since the only difference between them is the contribution of the root property. The root property used in NM2 can be computed as e^a , so there is a possibly optimal ratio between NM1 and NM2. We can estimate a maximum value of the best root property, set it as the ratio, and will not miss any possible good polynomial.

We choose $e^{-6.5} \approx 1/665$ as the ratio between NM1 and NM2, so NM2 is about 1.3×10^{19} and 6.3×10^{18} corresponding to NM1 in the above table respectively.

5.4 External Parameter Selections

Besides the parameters inside the Pol5 program, the most important parameter we need to decide is how much time we have to spend on it, or to say: "How many leading coefficients we have to search for?" We made a experiment on the optimal leading coefficients and discovered that it ranged from 5.5×10^7 to 2.5×10^8 . For the sake of safety, we select the upper-bound of leading coefficient (a_5) as 3×10^8 .

Number	Optimal a_5 Value
RSA-155	83772000
D155_01	56847420
D155_02	106560360

D155_03	255172680
D155_04	218225280
D155_05	90300420
D155_06	91654080

5.5 Overall Performance

Now we are able to test how much time we need to select a polynomial for a 155-digit number.

Number	Total Time (hrs)
RSA-155	1176.5
D155_01	1098.2
D155_02	984.5
D155_03	1117.2
D155_04	1154.5
D155_05	1167.8
D155_06	1114.0

$$a_5\text{-max} = 3 \times 10^8, \text{NM} = 7.12 \times 10^{23}$$

$$\text{NM1} = 4.20 \times 10^{21}, \text{NM2} = 6.30 \times 10^{18}$$

From the above table, we can see that it is possible to select a good polynomial for a 155-digit number in 24 hours with 50 cores, or in 3 hours with 400 cores.

Compared with the pair of polynomials used in the factoring record [4] and the one published in [6], the following pair of polynomials found at NTU is 56% and 6.8% better respectively.

$$\begin{aligned}
 f &= 83772000 x^5 \\
 &\quad - 55340006499600 x^4 \\
 &\quad - 57899874664053626478 x^3 \\
 &\quad + 4276456028202163925479457 x^2 \\
 &\quad + 235007922529884205334401821800 x \\
 &\quad - 1406850163218854524430305284200079 \\
 g &= 2054098293316505557 x \\
 &\quad - 167185341081359137443707501330
 \end{aligned}$$

Polynomial	Murphy_E
Record [4]	1.91×10^{-12}
Kleijung [6]	2.79×10^{-12}
Pol5	2.98×10^{-12}

6 Sieving

6.1 Algorithm Description

In order to proceed on the matrix reduction step, the number of relations found must exceed the number of primes in the factor base. Therefore, the goal of the sieving step is to find as many relations as possible in the shortest time. Since each relation can be represented by an (a, b) pair, sieving can be considered as searching for smooth (a, b) pairs on the (a, b) plane. The two sieve algorithms we used are the *line sieve* and the *lattice sieve* [13].

The line sieve is a naïve way of sieving. It simply searches the whole (a, b) plane horizontally, starting from $b = 1, 2, \dots$. This algorithm is quite effective when b is small, but the probability of (a, b) being smooth as b increases decreases quickly. Hence the amount of relations it can find is limited.

The lattice sieve, invented by Pollard [7], is a faster algorithm that found more than 99% of the relations in our experiments. Instead of naïvely sieving the whole (a, b) plane, the lattice sieve sieves only those (a, b) pairs whose $N(a - b\alpha)$ are divisible by a special q , which is a prime in the algebraic factor base. Let the special $q = (p, s)$, then the (a, b) pairs sieved on are those who satisfy $a - bs \equiv 0 \pmod{p}$. These (a, b) pairs form a lattice on the (a, b) plane that can be generated by two vectors $(p, 0)$ and $(s, 1)$. Utilizing the two vectors generating the lattice, we sieve only on (a, b) that are a part of the lattice.

Since the number of relations found by a single special q is limited, we have to try a range of special q 's in order to find enough relations.

Parallelization is extremely simple with lattice sieve. Neither MPI nor OpenMP is necessary, since all we have to do is to partition the sieve range of special q to many non-overlapping ranges, and each processor can sieve independently using the special q 's in the range given respectively.

6.2 GGNFS Implementation Details

The implementation of the above two algorithms can be found in the GGNFS package. The line sieve is implemented by Jason Papadopoulos, and the lattice sieve is implemented by Jens Franke [5].

Also in the GGNFS package are two programs that process the relations found by the sieve programs and tries to construct the matrix which will be solved in the matrix reduction step. The first program is *procrels*, used to organize the raw output of the lattice sieve. The second program, *matbuild*, attempts to construct the matrix with the relations available. The sieving step ends when enough relations are found and the matrix is successfully created.

6.3 Statistics of Our Experiment

The sieving step was conducted on the HP cluster. Due to constraints on the cluster, we could use only 50 cores simultaneously during most of experiments. However, since sieving is a very easily parallelized step, the time needed when we use more cores can be calculated by simple division plus a little overhead.

The range of special q sieved is [1000000, 53500000). The rational factor base consists of primes up to 2×10^7 , and the bound of algebraic factor base is 4×10^7 . Sieving took a calendar time of 76 hours and 13 minutes on 50 cores. The total time spent on sieving is 147 days 22 hours and 46 minutes.

69523978 relations which may have at most 4 large primes up to 2^{30} were found, and these relations were combined to form 4245955 relations without large primes.

Statistics Comparison between Ours and [4]		
	Ours	[4]
RFB Line Sieve_1 ₍₁₎	20 000 000	44 000 000 ⁽²⁾
AFB Line Sieve_1	40 000 000	110 000 000 ⁽²⁾
RFB Line Sieve_2	20 000 000	8 000 000 ⁽³⁾
AFB Line Sieve_2	40 000 000	25 000 000 ⁽³⁾
RFB Lattice Sieve	20 000 000	16 777 216 ⁽⁴⁾
AFB Lattice Sieve	40 000 000	16 777 216 ⁽⁴⁾
% Relations Found by Line Sieve	< 1%	29%
% Relations Found by Lattice Sieve	> 99%	71%
Calendar Time	76 hours 13 minutes	3.7 months
Total CPU Time	147 days 22 hours	35.7 years
Total Relations (no duplicates)	69 523 978	85 534 688
Large Primes Limit	1 073 741 824	1 000 000 000

- (1) RFB stands for rational factor base. AFB stands for algebraic factor base. For line sieve, there were two different factor base parameters used, which are labeled 1 and 2 respectively.
- (2) Two large primes on each side.
- (3) Two large primes on the rational side while three large primes on the algebraic side.
- (4) Two large primes on each side.
- (5) Computers used in [4]:
 - a) 160 SGI and Sun workstations (175-400 MHz)
 - b) 8 SGI Origin 2000 processors (300 MHz)
 - c) about 120 Pentium II PCs (300-450 MHz)
 - d) 4 Digital/Compaq boxes (500 MHz)

Statistics for *matbuild* is given in the next section.

7 Matrix Reduction

The matrix reduction step of the NFS is to solve the linear system $\mathbf{B}\mathbf{x} = 0$ over $\mathbf{GF}(2)$, where $\mathbf{B}$ is a huge sparse matrix obtained from the previous step. The solution $\mathbf{x}$ is the required dependency.

In our factoring of 512-bit RSA modulus, the matrix $\mathbf{B}$ has size about 3.4×10^6 by 3.4×10^6 ($= n$). It is too slow to use the direct Gaussian elimination, which has an $O(n^3)$ run time. To take the advantage of the sparsity of $\mathbf{B}$, iterative methods such as Lanczos or Wiedemann algorithm are applied.

7.1 Pruning

In order to reduce the matrix size so that it is faster to solve the system of linear equations, the matrix is usually “pruned” first [2]. GGNFS [14] has the program *matprune* doing the pruning work.

This part needs a lot of memory and is hard to parallelize. Besides, it is fast compared with the system solving step, so it is done by one processor for small matrices in GGNFS. However, as the size of the matrix increases, this non-parallel part becomes a bottleneck, so we need to do something to improve the efficiency.

We parallelize this part to get better performance. Details are discussed in section 7.3. Here is the effect of pruning:

	Matrix Size	Weight
Before Pruning	3703230×4245939	574694874
After Pruning	3420140×3438720	449074181

7.2 Block Lanczos

Lanczos algorithm [8] is very useful in solving a large sparse linear system. Montgomery [8] provides a block version of Lanczos algorithm, which takes $O(dn^2/k) + O(n^2)$ time, where d is the average weight of a column and k is the block size of the matrix.

Block Lanczos can solve the system $\mathbf{A}\mathbf{x} = 0$ where $\mathbf{A}$ is a symmetric matrix only. Let $\mathbf{A} = \mathbf{B}^T\mathbf{B}$, then $\mathbf{A}$ is symmetric. So we can apply Block Lanczos solving $\mathbf{A}\mathbf{x} = 0$, hoping $\mathbf{x}$ is also the solution to $\mathbf{B}\mathbf{x} = 0$.

Let k denote the computer word size, typically 32 or 64. It solves the system in the following way:

1. $\mathbf{X}_0 = 0$. $\mathbf{V}_0 = \mathbf{W}_0 = \mathbf{Y}$, a random $n \times N$ matrix.
2. Proceed through the i th Lanczos iteration:

$$\mathbf{V}_{i+1} = \mathbf{A}\mathbf{V}_i\mathbf{S}_i^T + \mathbf{V}_i\mathbf{D}_{i+1} + \mathbf{V}_{i-1}\mathbf{E}_{i+1} + \mathbf{V}_{i-2}\mathbf{F}_{i+1}$$

$$\mathbf{D}_{i+1} = \mathbf{I}_k - \mathbf{W}_i^{\text{inv}}(\mathbf{V}_i^T\mathbf{A}^2\mathbf{V}_i\mathbf{S}_i\mathbf{S}_i^T + \mathbf{V}_i^T\mathbf{A}\mathbf{V}_i)$$

$$\mathbf{E}_{i+1} = -\mathbf{W}_{i-1}^{\text{inv}}\mathbf{V}_i^T\mathbf{A}\mathbf{V}_i\mathbf{S}_i\mathbf{S}_i^T$$

$$\mathbf{F}_{i+1} = -\mathbf{W}_{i-2}^{\text{inv}}(\mathbf{I}_k - \mathbf{V}_{i-1}^T\mathbf{A}\mathbf{V}_{i-1}\mathbf{W}_{i-1}^{\text{inv}})$$

$$(\mathbf{V}_{i-1}^T\mathbf{A}^2\mathbf{V}_{i-1}\mathbf{S}_{i-1}\mathbf{S}_{i-1}^T + \mathbf{V}_{i-1}^T\mathbf{A}\mathbf{V}_{i-1})\mathbf{S}_i\mathbf{S}_i^T$$

$$\mathbf{W}_i^{\text{inv}} = \mathbf{S}_i(\mathbf{S}_i^T\mathbf{V}_i^T\mathbf{A}\mathbf{V}_i\mathbf{S}_i)^{-1}\mathbf{S}_i^T$$

Select $\mathbf{S}_i$ such that:

$\mathbf{W}_i = \mathbf{V}_i\mathbf{S}_i$ and $\mathbf{W}_i^T\mathbf{A}\mathbf{W}_i$ invertible;

$\text{rank}(\mathbf{W}_i)$ as large as possible;

Any column of $\mathbf{V}_{i-1}$ not in $\mathbf{W}_{i-1}$ must be used.

$\mathbf{X}_i = \mathbf{X}_{i-1} + \mathbf{V}_i\mathbf{W}_i^{\text{inv}}\mathbf{V}_i^T\mathbf{V}_0$

Terminate when $\mathbf{V}_m^T\mathbf{A}\mathbf{V}_m = 0$ for some m .

3. After iteration:

Let $\mathbf{Z} = \mathbf{X}_m + \mathbf{Y}$.

If $\mathbf{V}_m = 0$, then find the solution to $\mathbf{A}\mathbf{Z} = 0$.

If $\mathbf{V}_m \neq 0$, then using Gaussian elimination, one can take linear combinations of $\mathbf{V}_m$ and $\mathbf{Z}$ to find the solution.

GGNFS [14] implemented Montgomery’s Block Lanczos algorithm in the program *matsolve*.

7.3 Parallelization

In the original GGNFS, the sieving portion is already fairly well distributable for a large cluster. The other parts are not. As the size of the problem increases, the running time of *matprune* and *matsolve* increased to the point that it was dominating the running time. We made things a little more parallel.

Ideally, running time with p processors is $1/p$ of what it is with one processor. However, there is always overhead as we must split and distribute the problem, then later recombine the results. The last part is the most significant overhead for *matprune*.

Let α be the time to do a single small test component, d the average column weight, k the size of each block of the result, and β the time needed to sort and collate a processed result. Assuming that we also parallelize the recombination process, the speedup is $p/(1 + \beta p/\alpha d)$. When $p > 20$, $\beta p/\alpha d$ is not negligible. We have done our best to reduce β by avoiding the use of conditional statements.

In block lanczos we need to do many matrix multiplications: $m \times n$ by $n \times k$, $k \times n$ by $n \times m$, $k \times n$ by $n \times k$, these we know how to parallelize well. Since recombination in this case is just doing streamed *xor* (as addition over $\mathbf{GF}(2)$), the speedup does get close to a factor of p .

Both pruning and block lanczos are iterative, and it is difficult to avoid inter-processor communications. As we are not expert in handling these issues, we avoided MPI and implemented our parallelizations in OpenMP only.

7.4 Our Experiment

Our tests are run using 24 cores on the IBM p595 at NTU. It takes 2.23 hours for *matprune* to reduce the matrix size from 3703230×4245939 to 3420140×3438720 ; then it takes 37.52 hours for *matsolve* to solve the remaining system using block lanczos.

Statistics Comparison between Ours and [4]		
	Ours	[4]
Matrix Building	7 hours 20 minutes ₍₂₎	N / A
Matrix Pruning	2 hours 14 minutes ₍₃₎	N / A
Matrix Building and Pruning	9 hours 34 minutes	1 month ₍₄₎
Matrix Solving	37 hours 31 minutes ₍₃₎	224 hours ₍₅₎

- (1) The times listed in the table are all calendar time.
- (2) Since we are not sure if we can stop sieving until the matrix is successfully built, the time needed for matrix building is counted in the sieving calendar time. Matrix building was carried out with one CPU on the HP cluster. The time taken by *procel*s is also included.

- (3) Carried out on 24 cores of the IBM p595 SMP.
- (4) Carried out on an SGI Origin 2000 computer.
- (5) Carried out on one CPU on the Cray C916 supercomputer.

8 Square Root

It is the final step of the GNFS. We have found the dependency of relations, and obtained y^2 and β^2 . By evaluating square roots y and β , we get the congruent squares $x^2 \equiv y^2 \pmod{N}$ with the fact $x = \varphi(\beta)$.

There is no problem to find the square root y , since the integer factorization of y^2 is already with us. But extracting the square roots of algebraic number β^2 is much more complicated.

The difficulty of finding a square root of the algebraic number β^2 is that we only have the corresponding factorization of *prime ideals*, not the algebraic numbers. Using brute-force to try all possible algebraic numbers wastes too much time.

There are some methods available:

1. If $\mathbf{Z}[\theta]$ is a UFD (unique factorization domain), it is easy to find a square root by a way similar to finding integer square root. But it works in Special Number Field Sieve (SNFS) only.
2. Brute-force method: Factorize the polynomial $X^2 - \beta^2$ directly.
3. Couveignes' method: Using properties of the *norm* of β^2 . It requires the odd degree.
4. Montgomery's square root method [9, 11].

GGNFS [14] implements the square root method of Montgomery in its *sqr*t program. The steps are:

1. Transform the product of prime ideals $\langle \beta^2 \rangle$ to a simpler form
2. Use lattice reduction in ideals to construct series of algebraic numbers $\{\alpha_i\}$, which are used to approximate β .
3. When the approximation error is small enough, compute the square root directly.

We ran *sqr*t program from three dependencies, each running with one CPU on the HP cluster. One succeeded in returning a non-trivial solution. The average time taken was 2 hours 59 minutes.

Statistics Comparison between Ours and [4]		
	Ours	[4]
Square Root	2 hours 59 minutes	45 hours 33 minutes ₍₁₎

- (1) Average time taken for 4 dependencies on 4 separate SGI Origin 2000 300MHz CPUs.

9 Conclusions

We have explained how 512-bit RSA modulus can be factored within 3 days at NTU. Although the execution time can be reduced a little bit further by various tricks of implementations, it is sufficient to show that using RSA-512 is very dangerous today.

Moore's Law says that the number of transistors that can be inexpensively placed on an integrated circuit, hence processing speed and memory capacity as well, doubles approximately every two years. The factoring record set in 1999 spent 6 months, so it is not a surprise to see any software implementation of GNFS takes $180 \times 2^{-9/2} \approx 8$ days to factor a 512-bit integer in 2008.

With full capacity of HP cluster at NTU, the period of 3 days for factoring is better than our expectation when we started the project. The progress comes from the improvements of GNFS algorithm and implementation of developed by many experts in these years, and our effort during the past year.

Acknowledgements

The authors are grateful to the encouragement of Prof. D. J. Guan at National Sun Yat-sen University; valuable assistance of Jason Chang at the Computer and Information Networking Center, NTU; helpful discussion and comments from Chia-Hsin Chen at IIS, Academia Sinica, Prof. Dan Bernstein of University of Illinois at Chicago, USA, and Prof. Tanja Lange at Technische Universiteit Eindhoven, Nederland.

References

- [1] Kazumaro Aoki, Jens Franke, Thorsten Kleinjung, Arjen K. Lenstra, and Dag Arne Osvik, *A Kilobit Special Number Field Sieve Factorization*, Asiacypt 2007, LNCS 4833, pp.1-12, 2007.
- [2] Stefania Cavallar, *Strategies in Filtering in the Number Field Sieve*, ANTS-IV Conference, LNCS 1838, pp. 209-231, 2000, <http://db.cwi.nl/rapporten/abstract.php?abstractnr=695>
- [3] Stefania Cavallar, Bruce Dodson, Arjen K. Lenstra, Paul Leyland, Walter Lioen, Peter L. Montgomery, Brian Murphy, Herman te Riele, and Paul Zimmermann, *Factorization of RSA -140 Using the Number Field Sieve*, Asiacypt 1999, LNCS 1716, pp. 195-207, 1999.
- [4] Stefania Cavallar, Bruce Dodson, Arjen K. Lenstra, Walter Lioen, Peter L. Montgomery, Brian Murphy, Herman te Riele, Karen Aardal, Jeff Gilchrist, Gerard Guillerm, Paul Leyland, Joel Marchand, Francois Morain, Alec Muffett, Chris and Craig Putnam, and Paul Zimmermann, *Factorization of a 512-Bit RSA Modulus*, Eurocrypt 2000, LNCS 1807, pp. 1-18, 2000.
- [5] Jens Franke and Thorsten Kleinjung, *Continued fractions and lattice sieving*. SHARCS 2005, <http://www.ruhr-uni-bochum.de/itsc/tanja/SHARCS/talks/FrankeKleinjung.pdf>
- [6] Thorsten Kleinjung, *On Polynomial Selection for the General Number Field Sieve*, Mathematics of Computation, vol. 75, no. 256, pp. 2037-2047, October 2006.
- [7] Arjen K. Lenstra and Hendrik W. Lenstra, Jr. (Editors), *The Developement of the Number Field Sieve*, Lecture Notes in Mathematics, vol. 1554, Springer-Verlag, 1993.
- [8] Peter L. Montgomery, *A Block Lanczos Algorithm for Finding Dependencies over GF(2)*, Eurocrypt 1995, LNCS 921, pp. 106-120, 1995.
- [9] Peter L. Montgomery, *Square roots of products of algebraic numbers*, <http://ftp.cwi.nl/pub/pmontgom/sqrt.ps.gz>
- [10] Brian A. Murphy, *Polynomial Selection for the Number Field Sieve Integer Factorization Algorithm*, Ph. D. Thesis, Australian National University, 1999.
- [11] Phong Nguyen, *A Montgomery-like square root for the number field sieve*, ANTS III, LNCS 1423, pp. 151-168, 1998.
- [12] Ronald L. Rivest, Adi Shamir, and Leonard M. Adleman, *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems*, Communications of the ACM, 21(2), pp. 120-126, 1978.
- [13] Roger A. Golliver, Arjen K. Lenstra, and Kevin S. McCurley, *Lattice Sieving and Trial Division*, First International Symposium on Algorithmic Number Theory (ANTS), LNCS 877, pp. 18-27, 1994.
- [14] *General Purpose Factoring Records*, <http://www.crypto-world.com/FactorRecords.html>
- [15] *GGNFS – A Number Field Sieve Implementation*, <http://www.math.ttu.edu/~cmonico/software/ggnfs> <http://www.groups.yahoo.com/group/ggnfs> <http://ggnfs.cvs.sourceforge.net/ggnfs>